

Final Project Proposal

Prediction to Prescription: Building an AI Agent

Team: 1 **Project Title:** Optimizing Twitch Sponsorship Spend for Conversions **Date:** 31 May 2026

Name	Email	Program	Primary Role
Member 1 Shubh Vaishnav	vaishnav@purdue.edu	MSBAIM	Project Owner
Member 2 Nick Sopcic	nsopcic@purdue.edu	MSBAIM	Component owner
Member 3 Ethin DalPian	edalpian@purdue.edu	MSBAIM	Data Collection & Preparation
Member 4 Matt Foor	foor0@purdue.edu	MSBAIM	Component owner

Section 1 — Problem Statement and Business Context

Describe the class of business problem your agent will tackle — in plain language, without writing down a formal optimization model. Your agent will generate the problem formulation; your job here is to give it a clear, well-motivated problem to work on. Aim for 1–1.5 pages across the three subsections below.

1.1 Business Context

Our agent is designed for a game studio that markets through Twitch creators. Creators run live streaming sessions, and the studio can sponsor a session at a cost in order to drive purchases from the creator’s audience. The marketing manager holds a fixed sponsorship budget each period and must decide which creators to sponsor and how much to spend, with the goal of maximizing the purchases that sponsorship actually causes. Today the studio sponsors roughly half of available sessions with no systematic targeting, so spend is spread across creators regardless of how much incremental purchasing each sponsorship drives. High reach creators tend to get funded because they look impressive, even when their audience would have purchased anyway, and the manager has no reliable way to separate that baseline buying from the genuine lift produced by paying for the sponsorship.

1.2 Problem Class

This is a *budget allocation problem with diminishing returns* applied to advertising spend. Each period the manager chooses how much sponsorship budget to direct at each creator segment (continuous and non negative). The goal is to maximize the incremental conversions, the purchases caused by sponsorship rather than the purchases that would have happened regardless. The decision is limited by a fixed total budget and optional per segment spending caps. The uncertainty lies in the response curves, the relationship between sponsorship spend and incremental conversions for each segment, which is unknown and must be estimated from data. The problem is concave because each segment saturates: paying more lifts conversions at a decreasing rate. A discrete variant of

the same problem, choosing which individual creators to sponsor under a budget, is a selection (knapsack) problem, and our agent supports both framings.

1.3 Why This Problem Is Hard Without an Agent

The problem is hard without an agent for three reasons. First, the effect of sponsorship is confounded and noisy: purchases occur with or without sponsorship, so a manager cannot eyeball the incremental lift per dollar from the raw numbers, and the creators who get sponsored are not chosen at random. Second, the constraints interact: a single budget must be split across many creators whose marginal effectiveness differs and diminishes with spend, which is a nonlinear resource allocation problem rather than a ranking. Third, the situation changes constantly: the creator roster, audiences, and costs shift period to period, so any fixed split goes stale and manual reoptimization is impractical. The agent will outperform current practice by estimating an incremental response curve for each segment, solving the constrained allocation exactly, and adapting as new session data arrives. Success will be measured by higher incremental conversions on held out creators and sessions compared with the studio’s current sponsorship pattern.

Section 2 — Data Plan

Identify every real dataset you will use. For each one, provide the source, approximate size, key features, and how it connects to the uncertain parameters in your model. Explain your train/test split strategy. Aim for 0.5–1 page.

2.1 Primary Dataset

Field	Details
Dataset name	Twitch creator session data (<code>twitch_df</code>)
Source	Course provided
Approximate size	39,928 sessions across 2,000 creators, 16 columns
Key features	Session timing (day of week, start hour, duration), audience (unique viewers, average viewers, five minute viewers, creator followers), creator gender; sponsorship treatment (<code>sponsored</code> , <code>sp_cost</code> in USD, present only when sponsored)
Outcome variable	<code>purchases_3h</code> (purchases in the three hours after a session); <code>purchases_1h</code> used as a robustness check
Known limitations	Sponsorship is not randomized, so creators self select into being sponsored and incremental estimates must adjust for observed confounders; <code>sp_cost</code> exists only for sponsored sessions; the post session <code>active_users</code> fields are outcomes, not inputs, and are excluded as predictors

2.2 Additional Datasets

We also use the player game dataset, a training file of 12,000 players and a held out test file of 3,000 players described by the same codebook, whose prediction period dependent variables are `dv_purchase_amt` and `dv_churn`. There is no row level key linking individual players to creators, so we do not merge the two sources. Instead, this dataset supplies the economic value of a conversion: a small value model estimates the expected purchase amount per converting user, with an optional churn adjusted retention factor, and that value is used to weight predicted Twitch conversion counts into expected revenue. This lets the optimizer trade off spend against revenue rather than against raw purchase counts while keeping Twitch conversions as the central outcome.

2.3 Train / Test Split

For the Twitch data we use a grouped split on `creator_id` so that no creator appears in both training and test. Creator level effects are strong, and an ordinary random split would leak a creator’s behavior across the split and inflate our scores. We hold out a set of whole creators as the final test set and run grouped k fold cross validation on the remaining creators for model selection and hyperparameter tuning. For the player value model we use the provided 12,000 row training set with k fold cross validation and predict on the 3,000 row held out test set.

Section 3 — Methods Sketch

Outline the technical approach for each agent component. You do not need final answers — but you must show end-to-end thinking from data input to decision output. Address prediction, optimization, constraint handling, and baseline. Aim for 1–1.5 pages.

3.1 Prediction Layer

The optimizer needs a smooth, concave curve of incremental conversions against sponsorship spend for each creator segment, while the raw data is a purchase count recorded per session together with whether and how much that session was sponsored. We keep a two stage hybrid design.

Stage 1: Session conversion and uplift model. We train a gradient boosted regressor (XGBoost) to predict post session purchases from pre decision features only: creator followers, unique, average, and five minute viewers, duration, day of week, start hour, creator gender, the sponsorship indicator, and `sp_cost`. Because purchases are large non negative counts, we fit a count appropriate loss such as Poisson or Tweedie deviance, or squared error on log purchases, rather than plain squared error, and we exclude the post session `active_users` fields because they are outcomes rather than inputs. To recover the incremental effect of sponsorship rather than a raw prediction, we use an uplift framing: incremental conversions at spend x are the predicted purchases when a session is sponsored at x minus the predicted purchases for the same session left unsponsored. We will compare an S learner (a single model with the treatment as a feature) against a T learner (separate sponsored and unsponsored models) and keep whichever produces more stable lift estimates. Because sponsorship is not randomized, the lift is adjusted for observed creator and session

characteristics, and we are explicit that residual unmeasured confounding is a limitation rather than claiming a clean causal effect.

Stage 2: Segment response curves. We group creators into segments, for example by follower tier and typical audience size, aggregate the Stage 1 incremental predictions to the segment level, pair sponsorship spend with incremental conversions, and fit a concave saturation curve for each segment s . We use a Hill saturation form, $g_s(x) = \frac{V_s x^{n_s}}{K_s^{n_s} + x^{n_s}}$, with a logarithmic curve as a fallback, fit by nonlinear least squares using `scipy.optimize.curve_fit`. The fitted parameters V_s (saturation ceiling) and K_s (half saturation point) are the uncertain inputs the optimizer searches over. The value model from Section 2.2 supplies an expected value per conversion, so the same curves can be expressed in incremental revenue as well as in incremental conversions.

A single XGBoost model on its own is a poor fit for the prescription step: a tree ensemble outputs a step function that is non concave and non monotone, so feeding it to the allocator yields unstable spend that sits on flat or jagged regions of the surface. The parametric Stage 2 curve is what makes the allocation well posed, so we let XGBoost do the prediction and let the parametric layer supply a clean concave object for optimization.

On the choice of loss, a count appropriate loss in Stage 1 and nonlinear least squares in Stage 2 are sufficient because the concave Stage 2 curve already aligns the prediction target with the downstream concave objective, and the modular seam keeps the pipeline transparent and easy to reparameterize for the Section 5.2 stakeholder change. A fully decision aware (SPO style) objective is noted as a stretch goal if time permits.

3.2 Optimization Engine

We allocate the sponsorship budget across creator segments with `scipy.optimize.minimize` using Sequential Least Squares Programming (SLSQP), maximizing total incremental conversions (or incremental revenue), $\sum_s g_s(x_s)$, where each g_s is the concave Stage 2 curve; because the solver minimizes, we pass it the negative of this sum. The summed objective is concave and the feasible region is convex, so the problem has a single global optimum in principle, and we wrap the solver in a multistart loop that runs from several random feasible starting allocations and keeps the best objective, which protects against numerical flat spots and any mild non concavity from an imperfect curve fit. The same agent supports the discrete variant, choosing which individual creators to sponsor under the budget, which is a knapsack problem; in that mode we switch from SLSQP to an integer program built with PuLP. Which mode runs is a field in the problem schema (Section 5.2). We keep CVXPY in reserve as a convex check on the continuous mode: when the fitted curves are cleanly concave, CVXPY should reproduce the SLSQP optimum, which validates both layers.

3.3 Constraint Handling

We enforce every constraint as a hard constraint inside the solver rather than through penalty terms, so each allocation the agent proposes is feasible by construction. The continuous mode has four. First, a budget constraint, total sponsorship spend at most the period budget, $\sum_s x_s \leq B$, written as an inequality because spending less is allowed once a segment’s marginal lift is exhausted. Second, per segment caps, $0 \leq x_s \leq u_s$, passed as box bounds to prevent over concentration on a single segment. Third, non negativity, which the lower bound guarantees. Fourth, a minimum sponsorship size where a segment is funded, since observed `sp_cost` floors near \$500 and a funded

segment must clear that floor. The discrete mode adds binary selection constraints, which SLSQP cannot express directly and which the integer solver handles instead. We prefer hard constraints over penalties because penalties require tuning a weight, can leave small residual violations, and would muddy the explanation the agent gives the stakeholder.

3.4 Baseline Comparison

The current practice we aim to beat is the studio’s existing sponsorship pattern, roughly half of sessions funded with no incrementality targeting. We compare against two baselines: the observed historical allocation, and a reach chasing heuristic that spends on the highest follower creators first. Both are scored on the same held out creators with the same fitted response curves, so the only thing that differs is the allocation rule. Improvement is measured as the increase in incremental conversions, and in incremental revenue, of the agent’s allocation over each baseline. Beating the reach chasing heuristic is the meaningful test, because chasing reach is precisely the mistake the manager makes today when audience size is confused with the lift that sponsorship actually buys.

3.5 Pipeline Diagram

The data to decision flow has five stages. Session and creator records feed the Stage 1 conversion and uplift model, whose incremental estimates are aggregated and fit into concave segment response curves, which become the parameters the optimizer searches over, producing a sponsorship allocation together with a plain language explanation. The player value model feeds the objective so that conversions can be weighted by expected revenue.

Session + creator data → **XGBoost uplift model (Stage 1)** → **Segment response curves**
(Stage 2) → **scipy multistart optimizer** → **Allocation + explanation**

Section 4 — Team Roles

Assign each member a primary component. Every one of the five components must be covered. Roles are not binding, but each member must have clear ownership.

Component	Primary Owner	Brief Description of Responsibility
Problem Formulation Module	Shubh Vaishnav	Translates the sponsorship problem into decision variables, objective, and constraints, and maintains the problem schema including the continuous versus discrete decision mode.
Data Collection & Preparation	Ethin DalPian	Cleans the Twitch and player data, engineers pre decision session features, builds the creator segments, and derives the per conversion value from the player data.
Prediction Layer	Nick Sopcic	Builds the two stage predictor, the XGBoost conversion and uplift model and the concave segment response curves passed to the optimizer.
Optimization Engine	Matt Foor	Implements the scipy multistart solver and the integer alternative, encodes the budget and segment constraints, and returns the optimal allocation.
Explanation & Validation	Shubh Vaishnav	Runs the bootstrap uplift test against the baselines and produces the plain language rationale the agent reports to the stakeholder.

Of course our team will collaborate with each other throughout the project so shared ownership is always implied.

Section 5 — Benchmark and Validation Plan

Explain how you will evaluate success on (a) your own problem and (b) the instructor benchmark. For (a), state the metric and how you will interpret it. For (b), explain how your agent can accept a new problem description as input. Aim for 0.5 pages.

5.1 Evaluating Your Own Problem

Our primary metric is incremental conversions on the held out creators, with incremental revenue reported alongside it through the value weight. We compare the agent’s allocation against the two baselines from Section 3.4 on the same held out creators and the same fitted curves, so any difference reflects the allocation rule alone. Because a single point estimate of uplift can be noisy, we establish success through a bootstrap over held out creators: we resample creators with replacement many times, recompute the conversion uplift of the agent over each baseline on every resample, and build

a confidence interval for that uplift. We treat the result as a meaningful win when the lower bound of the interval stays above zero, meaning the agent beats the baseline consistently rather than on a lucky split, rather than fixing an arbitrary percentage threshold in advance. We report the median uplift, the interval, and the realized allocation so the size and direction of the spending shift are visible to the stakeholder.

5.2 Stakeholder Robustness

Our agent is built so that the business reality lives in data, not in code. The pipeline reads a structured problem specification, and the model and solver are written against that specification rather than against fixed numbers.

- **What is parameterized.** The specification holds the period budget, the per segment caps, the segment definitions, the cost assumptions, the objective (conversions versus revenue, total versus incremental), and the decision mode (continuous spend across segments versus discrete selection of creators). The response curves, the constraint set, and the objective passed to the solver are all built from these fields at run time. Only the structural choices, the two stage architecture, the concave curve family, and the model classes, are fixed in code.
- **How a new problem reaches the agent.** We use two layers. A human can edit the specification directly as a configuration dictionary, which is fast and unambiguous. In parallel, an LLM reads the instructor’s natural language problem change and emits the same specification, so a plain English instruction such as a new cap on a creator tier, a switch to revenue, or a switch from continuous spend to discrete selection is translated into the structured fields automatically. A validation step checks the LLM output against the schema, the allowed ranges, and feasibility before anything reaches the solver, so a malformed or contradictory change is caught rather than silently solved.
- **What the limits are.** Changes expressed in the existing vocabulary, a changed budget, a new or tightened cap, a switch between supported objectives, or a switch between the continuous and discrete modes, require no code change and solve again in seconds. Changes outside that vocabulary need real work. A brand new decision dimension, a non concave objective that breaks the continuous solver, or a constraint that needs data we do not have would require rebuilding the prediction or optimization layer, not just a new specification. We will be explicit with the stakeholder about which bucket a requested change falls into and what the business cost of the resulting reallocation is.

References (if any)

- [1] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- [2] Kunzel, S. R., Sekhon, J. S., Bickel, P. J., & Yu, B. (2019). Metalearners for Estimating Heterogeneous Treatment Effects Using Machine Learning. *Proceedings of the National Academy of Sciences*, 116(10), 4156–4165.

- [3] Jin, Y., Wang, Y., Sun, Y., Chan, D., & Koehler, J. (2017). *Bayesian Methods for Media Mix Modeling with Carryover and Shape Effects*. Google Inc.
- [4] Course provided datasets: Twitch creator session data and player game data, with accompanying codebook (MGMT 590-037, Summer 2026).